Лабораторная работа №1 - Настройка окружения и основы NumPy

Цель

Освоить инструменты рабочего окружения аналитика данных (Jupyter Notebook, Google Colab) и научиться выполнять векторизованные вычисления с использованием библиотеки NumPy, понимая их преимущество перед стандартными структурами данных Python.

Задачи работы

Научиться:

- разворачивать и настраивать рабочее окружение для анализа данных;
- работать с ячейками Jupyter Notebook (код, markdown, magic-команды);
- создавать и модифицировать массивы NumPy;
- применять индексацию, срезы и маски для отбора данных;
- использовать механизм broadcasting;
- выполнять базовые линейно-алгебраические операции;
- сравнивать производительность вычислений на «чистом» Python и на NumPy.

Методические указания к выполнению лабораторной работы

Занятие 1. Настройка окружения и знакомство с Jupyter Notebook

Часть 1. Настройка окружения

Задание 1.1. Установить Anaconda/Miniconda (либо зарегистрироваться в Google Colab). Создать новую записную книжку (notebook) и сохранить её под указанным именем.

Задание 1.2. Создать виртуальное окружение `data_tools_lab` (при локальной установке) и убедиться, что в нём доступны Python и pip:

```
conda create -n data_tools_lab python=3.10
```

```
conda activate data_tools_lab
pip install numpy jupyter
```

Часть 2. Знакомство с Jupyter Notebook

Задание 1.3. Изучить и продемонстрировать в отчёте:

- разницу между ячейками типа Code и Markdown;
- сочетания клавиш Shift+Enter, Esc, A/B (вставка ячеек), dd (удаление);
- как минимум 3 magic-команды: `%timeit`, `%%time`, `%who` (или `%whos`).

Задание 1.4. Оформить в markdown-ячейке титульный блок отчёта (ФИО, группа, тема, дата).

Часть 3. Первое знакомство с NumPy

Задание 1.5. Импортировать NumPy. Создать:

- одномерный массив из списка [1, 2, 3, 4, 5];
- двумерный массив 3×3 из чисел от 1 до 9;
- массив нулей 5×5, массив единиц 3×4, единичную матрицу 4×4.

Для каждого массива вывести: `shape`, `dtype`, `ndim`, `size`.

Задание 1.6. Создать массив из 20 случайных целых чисел от 0 до 100 (`np.random.randint`). Найти его минимум, максимум, среднее, медиану и стандартное отклонение без использования циклов.

Занятие 2. Индексация, broadcasting, сравнение производительности

Часть 1. Индексация, срезы, маски

Задание 2.1. На массиве из задания 1.6:

- получить элементы с индексами от 3 до 10;
- получить каждый второй элемент;
- получить все элементы, кратные 5, с помощью булевой маски;
- заменить все значения меньше 20 на 0.

Задание 2.2. Создать двумерный массив 5×5 со случайными числами. Вывести:

- вторую строку целиком;
- третий столбец целиком;
- подматрицу из первых двух строк и трёх столбцов;
- элементы главной диагонали.

Часть 2. Broadcasting и операции с массивами

Задание 2.3. Создать массив температур в Цельсиях (10 значений) и перевести его в Фаренгейты без использования циклов, используя `broadcasting (F = C * 9/5 + 32)`.

Задание 2.4. Создать матрицу 4×3 и вектор из 3 элементов. Продемонстрировать, как вектор автоматически «растягивается» при поэлементном сложении с каждой строкой матрицы. Объяснить результат в текстовом выводе.

Задание 2.5. Выполнить базовые линейно-алгебраические операции:

- транспонирование матрицы;
- матричное умножение двух матриц подходящих размерностей (`np.dot` или оператор `@`);
- вычисление определителя квадратной матрицы (`np.linalg.det`).

Часть 3. Сравнение производительности

Задание 2.6. Реализовать одну и ту же задачу двумя способами — на «чистом» Python (циклы, списки) и на NumPy (векторизация). Для каждой из следующих операций измерить время выполнения через `%timeit` на массиве из 1 000 000 элементов и оформить сравнение в виде таблицы в markdown-ячейке:

- поэлементное сложение двух массивов;
- вычисление суммы всех элементов;
- вычисление квадратов всех элементов массива.

Итоговый вывод: сформулировать (2–4 предложения), почему NumPy показывает такой прирост производительности.

Контрольные вопросы

1. Чем массив NumPy отличается от списка Python с точки зрения хранения данных в памяти?
2. Что такое broadcasting и при каких условиях он применим?
3. В чём разница между `np.dot(a, b)` и поэлементным умножением `a * b`?
4. Почему векторизованные операции выполняются быстрее циклов на чистом Python?
5. Что покажет `arr.dtype`, если массив создан из смешанного списка [1, 2.5, 3]?
6. Как получить копию массива, а не ссылку на исходные данные при срезе?

Требования к защите

На защите необходимо наличие проекта и отчета. Отчет по лабораторной работе делается в электронном виде (структура – титульный лист, задание, выполнение со скриншотами, вывод), пользоваться им на защите нельзя. Отчет загружается в Moodle.

Проект оформляется в виде единого Jupyter Notebook (файл .ipynb), включающего:

- markdown-заголовки к каждому заданию;
- код решения;
- вывод результатов выполнения ячеек (notebook сдаётся выполненным, без ошибок);

Файл именуется по шаблону: №Group_Familia_IO_LR1.ipynb.

Комментарии должны отсутствовать как в коде, так и в отчёте. Тоже самое касается подсказок в коде. Наличие комментариев и подсказок будет означать, что работа не сдана. Ее можно сдать только на следующую пару!!!

Рекомендуемые источники

1. Официальная документация NumPy: https://numpy.org/doc/stable/
2. VanderPlas J. Python Data Science Handbook — глава «Introduction to NumPy» (свободно доступна онлайн).
3. Документация Jupyter: https://jupyter.org/documentation

Таблица 1 – Технический стек

Инструмент	**Назначение**	**Ссылка**
Python 3.10+	язык программирования	python.org
Anaconda / Miniconda	дистрибутив с пакетным менеджером	anaconda.com
Google Colab (альтернатива)	облачная среда без установки	colab.research.google.com
Jupyter Notebook/Lab	интерактивная среда разработки	входит в Anaconda
NumPy	библиотека для работы с массивами	numpy.org